



Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

Fase 2

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Willy Ramírez, Alison Ramírez, Pablo Namicela, Aytana Salinas
Asignatura	Matemáticas Discretas
Ciclo	1 A
Unidad	1
Resultado de aprendizaje de la unidad	Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001 Fase 2
Tipo	Grupal
Título de la Práctica	Proposiciones, Conectores Lógicos, Inferencia, Deducción y Tablas de Verdad
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	06 de abril al 15 de mayo de 2026
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	18 horas (3 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Identificar y clasificar proposiciones lógicas simples y compuestas, distinguiendo entre proposiciones y enunciados no proposicionales.

- Comprender y aplicar los conectores lógicos fundamentales (negación, conjunción, disyunción, condicional, bicondicional) en la formulación de expresiones lógicas.
- Construir tablas de verdad para proposiciones compuestas, determinando su valor de verdad en todos los casos posibles.
- Aplicar las leyes de inferencia y deducción lógica para resolver problemas de razonamiento lógico en contextos computacionales y de ingeniería.
- Desarrollar habilidades de trabajo colaborativo, pensamiento crítico y resolución de problemas, integrando principios de equidad, inclusión y responsabilidad ética en el aprendizaje.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad
- Guía de ejercicios de inferencia y deducción lógica
- Calculadora científica
- Material bibliográfico de apoyo (textos digitalizados de referencia)
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entrega de evidencias
- Entorno virtual de aprendizaje (EVA-UNL)
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

FASE 2: Construcción de Tablas de Verdad y Operaciones Combinadas (Semana 2: 13-17 abril 2026)

1. El docente profundiza en la construcción de tablas de verdad para proposiciones compuestas con múltiples conectores, explicando el orden de evaluación (jerarquía de operadores) y el método sistemático para generar todas las combinaciones posibles de valores de verdad.
2. Los estudiantes construyen tablas de verdad para expresiones con 2, 3 y 4 variables proposicionales, identificando los casos en que la expresión resulta verdadera, falsa o es una tautología/contradicción/contingencia.
3. Práctica grupal: cada equipo resuelve ejercicios de simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, conmutativa, asociativa, distributiva, de Morgan, absorción).

4. Los equipos analizan problemas de lógica computacional: cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos en Python/C/Java).
5. Entrega del primer portafolio de ejercicios individuales (propuestas y tablas de verdad) a través de la plataforma EVA. Discusión sobre la ética en el uso de la lógica para la toma de decisiones algorítmicas y su impacto en la sociedad.

5. Resultados

- Portafolio de ejercicios individuales de proposiciones y tablas de verdad.
- Ejercicios resueltos de equivalencias lógicas y formas normales (FNC/FND).
- Mapa conceptual de las reglas de inferencia lógica.
- Argumentos lógicos analizados y demostrados con validez.
- Portafolio digital integrador con todos los productos de las 6 fases.
- Examen sumativo aprobado con nota mínima de 7/10.

Portafolio Digital

Willy Ramírez:

https://drive.google.com/drive/folders/1MUTrvZ2TMGCOYaloeWbcpgKZn56N1gnL?usp=drive_link

Alison Ramírez:

https://drive.google.com/drive/folders/1mU5eeP7leTygHdCdSztsgkJSFzObXsk3?usp=drive_link

Aytana Salinas:

https://drive.google.com/drive/folders/1jxUmpx0dob6t4pYgvyuLP6wgbRvE03Ju?usp=drive_link

Pablo Namicela:

https://github.com/pablonamicelaim/portafolio_DigitalMatem-ticasD.git

EJERCICIOS RESUELTOS POR LOS ESTUDIANTES

TABLAS DE VERDAD

SIMPLIFICACIÓN DE EXPRESIONES LÓGICAS

IF-ELSE, WHILE CON OPERADORES LÓGICOS

UNL 

- Estudiante: Alison Cristina Ramirez
- Asignatura: Matemáticas Discretas
- Título: Construcción de tablas de verdad
- Docente:
- Ciclo:
- Unidad:

FASE 2;

Item 2; Tabla de verdad (5 ejercicios)

- ① 2 variables

$(p \wedge q) \rightarrow p$ $2^2 = 4$ filas

p	q	$p \wedge q$	$(p \wedge q) \rightarrow p$
V	V	V	V
V	F	F	V
F	V	F	V
F	F	F	V

→ TAUTOLOGÍA

- ② $p \vee q \leftrightarrow \neg p \wedge \neg q$ $2^2 = 4$ filas

p	q	$\neg p$	$\neg q$	$\neg p \wedge \neg q$
V	V	F	F	F
V	F	F	V	F
F	V	V	F	F
F	F	V	V	V

→ Contradicción

- ③ 3 variables. → $(p \vee \neg q) \wedge r$ $2^3 = 8$ filas

q	(p	$\neg q$)	$\wedge$	r
V	V	F	V	V
V	V	F	F	F
F	V	V	V	V
F	V	V	F	F
V	F	F	F	V
V	F	F	F	F
F	F	V	V	V
F	F	V	F	F

→ CONTINGENCIA

- ④ $(p \vee q) \wedge \neg r$ $2^3 = 8$ filas

r	$(p \vee q)$	$\neg r$
V	V	F
F	V	V
V	V	F
F	V	V
V	F	F
F	F	V
V	F	F
F	F	V

→ CONTINGENCIA

- ⑤ 4 variables $[(p \vee q) \vee r] \wedge s$ $2^4 = 16$ filas

s	$[(p \vee q) \vee r]$	$\wedge$	s
V	V	V	V
V	V	F	F
V	V	V	V
V	V	F	F
V	V	V	V
V	V	F	F
V	V	V	V
V	V	F	F
V	V	V	V
V	V	F	F
V	V	V	V
V	V	F	F
V	V	V	V
V	V	F	F
V	V	V	V
V	V	F	F

→ CONTINGENCIA

Item 3. Simplificación de expresiones lógicas aplicando las leyes fundamentales (5 ejercicios)

• ① Demostrar

$$P \wedge P = P$$

$$P \wedge P = (P \wedge P) \vee 0 \rightarrow \text{Ley de identidad}$$

$$= (P \wedge P) \vee (P \wedge \neg P)$$

$$= P \vee (P \wedge \neg P) \rightarrow \text{Ley de Idempotencia}$$

$$= (P \vee P) \wedge (P \vee \neg P) \rightarrow \text{Ley distributiva}$$

$$= P \wedge 1 \rightarrow \text{Ley de identidad}$$

$$= P // Rta$$

• ② $[(P \vee Q \vee R) \vee (\neg P \wedge \neg Q \wedge \neg R)] \wedge [(Q \wedge R) \vee (Q \wedge \neg R) \vee (\neg Q \wedge R)] \leftrightarrow (Q \vee R)$

$$[\neg(\neg(P \vee Q \vee R)) \vee (\neg P \wedge \neg Q \wedge \neg R)] \rightarrow \text{Ley de Morgan}$$

$$= 1 \wedge [(Q \wedge R) \vee (Q \wedge \neg R) \vee (\neg Q \wedge R)] \rightarrow \text{Ley distributiva}$$

$$= [(Q \wedge 1) \vee (\neg Q \wedge R)] \rightarrow \text{Ley de complemento}$$

$$= [Q \vee (\neg Q \wedge R)] \rightarrow \text{Ley de identidad}$$

$$= [(Q \vee \neg Q) \wedge (Q \vee R)] \rightarrow \text{Ley distributiva}$$

$$= 1 \wedge (Q \vee R)$$

$$= (Q \vee R) // Rta$$

• ③ $(P \wedge Q) \vee (P \wedge \neg Q)$

$$= P \wedge (Q \vee \neg Q) \rightarrow \text{Ley distributiva}$$

$$= P \wedge 1 \rightarrow \text{Ley de complemento}$$

$$= P \cdot 1 \rightarrow \text{Ley identidad}$$

$$P // Rta$$

• ④ $(P \vee Q \vee R) \wedge (P \vee \neg Q \vee R)$

$$= (P \vee R \vee Q) \wedge (P \vee R \vee \neg Q) \rightarrow \text{Orden}$$

$$= (P \vee R) \vee Q \wedge (P \vee R) \vee \neg Q \rightarrow \text{Ley distributiva}$$

$$= (P \vee R) \vee (Q \wedge \neg Q) \rightarrow \text{Ley distributiva}$$

$$= (P \vee R) \vee 0 \rightarrow \text{Ley de complemento}$$

$$= P \vee R // Rta$$

• ⑤ $(P \vee \neg P) \wedge Q$

$$= 1 \wedge Q \rightarrow \text{Ley de complemento}$$

$$= Q // Rta$$

Item 4. Expresiones booleanas aplican, en la evaluación de condiciones en programación.

La expresión booleana permite la toma de decisiones en la programación evaluando las condiciones si es verdadero (True) o falso (False) para tomar control de la ejecución mediante estructuras condicionales.

edad = 20

```
if edad >= 18;
```

```
    printf("Eres mayor de edad");
```

```
else:
```

```
    printf("Eres menor de edad");
```

if-else: Evalúa si alguna expresión booleana es verdadera para luego ejecutar un bloque de código específico de lo contrario siendo falso omite el bloque.

While: Las expresiones booleanas son usadas para determinar si un bloque debe repetirse continuamente siempre y cuando la expresión sea evaluada como True.

Operadores lógicos: Sirve para conectar múltiples expresiones booleanas para formar condiciones compuestas se clasifican en tres:

And: Verdadero si todas son verdaderas

Or: Verdadero

Not: Se invierte el valor; si es 1 se convierte en 0 y viceversa.

```
boolean terminado = false;
```

```
int intentos = 0;
```

```
while (!terminado) {
```

```
    intentos ++
```

```
    if (intentos >= 3) {
```

```
        terminado = true // Cambia el estado para salir.
```

```
        System.out.println("Intento: " + intentos);
```

```
    }
```

Python: Usa las palabras clave True y False (mayúsculas) lo cual considera 0 y estructuras vacías como False.

C/Java: Usan true y false. En C, 0 es false y un número diferente de 0 es true. En Java los operadores son && (AND), || (OR), ! (NOT).

La expresión booleana permite la toma de decisiones en la programación, evaluando las condiciones si es verdadero(True) o falso (false) para tomar control de la ejecución mediante estructuras condicionales.

If-else: Evalúa si alguna expresión booleana es verdadera para luego ejecutar un bloque de código específico de lo contrario siendo falso omite

```
Python
edad = 20

if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

el bloque.

While: Las expresiones booleanas son utilizadas para determinar si un bloque debe repetir continuamente siempre y cuando la expresión sea evaluada como true.

Operadores lógicos: Sirve para conectar múltiples expresiones booleanas para formar condiciones compuestas, se clasifican en tres:

AND: Verdadero si todas las expresiones son verdaderas

OR: Verdadero si al menos una expresión es verdadera.

NOT: Se invierte el valor es decir verdadero pasa a falso o viceversa.

```
Python

numero = int(input("Introduce un número entre 1 y 100: "))

# El bucle sigue mientras el número esté FUERA del rango
while numero < 1 or numero > 100:
    print("Error: El número está fuera de rango.")
    numero = int(input("Inténtalo de nuevo: "))

print("¡Éxito! El número es válido.")
```

Python: Utiliza las palabras clave True y False (mayúsculas) lo cual considera 0 y estructuras vacías como False.

C / Java: Utilizan true y false (minúsculas). En C, 0 es false y un número diferente de 0 es true. En Java, los operadores son && (AND), || (OR), ! (NOT).

6. Preguntas de Control

1. Explique la diferencia entre una proposición simple y una proposición compuesta.

Generalmente una proposición simple expresa una idea básica y concreta donde no se presentan conectores mientras que las proposiciones compuestas es la unión de uno o más proposiciones simples, se da más de una idea e incluye conectores, también se puede desglosar en ideas más pequeñas. En el lenguaje de verdad son oraciones simples sin conectores y una proposición compuesta ya tiene conectores, varias ideas, ejemplo "**Si** Ana estudia **entonces** aprobará el examen" entonces es un conector ($p \rightarrow q$) es condicional

2. ¿Por qué es importante distinguir entre ambas en el contexto de la programación y diseño de algoritmos? Argumente con ejemplos concretos en lenguajes de programación.

Es importante distinguir porque permite garantizar la estructura de los algoritmos. Las proposiciones simples son afirmaciones indivisibles puede ser verdaderas (1) o falsa (0), las compuestas unen varias de estas mediante conectores lógicos y permite dar escenarios complejos dentro de una sola línea de código.

Lenguaje de programación en python

```
if edad >= 18:  
    print("Acceso permitido")
```

Lenguaje de programación en java

```
if (usuario.estaLogueado && usuario.esAdmin) {  
    mostrarPanelDeControl();  
}
```

7. Conclusiones

Se comprendió que las reglas de inferencia son herramientas fundamentales para automatizar el razonamiento lógico, permitiendo llegar a conclusiones válidas de forma sistemática.

La simplificación de funciones lógicas demostró ser una competencia clave para la ingeniería, impactando directamente en la eficiencia de la lógica de programación y el diseño de circuitos básicos.

8. Recomendaciones

Se sugiere el uso de diagramas de flujo lógicos para visualizar las premisas antes de proceder a la deducción formal, lo que facilita la identificación de la regla de inferencia adecuada.

Es recomendable practicar la conversión de lenguajes de programación de alto nivel a proposiciones lógicas para fortalecer la capacidad de depuración de código complejo.

9. Bibliografía / Referencias

Básica:



- Oscar Levin, University of Northern Colorado (2022). Discrete Mathematics: An Open Introduction – 4th Edition. Universidad Tecnológica de Pereira. ISBN: 9781534970748.
- Harris Kwon (2024). A Spiral Workbook for Discrete Mathematics. Milne Open Textbooks. ISBN: 978-1-956862-01-0.

Complementaria:

- André Greiner-Petter (2023). Making Presentation Math Computable. Springer. ISBN: 978-3- 658-40472-7.
- Kevin Powell (2025). Discrete Perspectives in Mathematics – Second Edition.
- Kazushi Ikeda et al. (2024). Advanced Mathematical Science for Mobility Society. Springer. ISBN: 978-981-99-9771-8.

Recursos de Internet:

- PLATZI. (2025). Curso de Matemáticas Discretas. Disponible en: <https://platzi.com>
- Juan Miguel León Rojas. Cuestiones Matemáticas Discretas. Disponible en: <https://www.ugr.es>
- Oscar Levin. Discrete Mathematics (Levin). Disponible en: <https://discrete.openmathbooks.org>

10. Anexos

Universidad Nacional de Loja.

APE1 U1 FASE 2

Nombre: Aytana Salinas.
Ciclo: 1A.
Unidad: 1.
Materia: Matemáticas Discretas.

Fase 2: Construcción de Tablas de Verdad y Operaciones Combinatorias. (Semana 2: 13-14 abril 2026).

ITEMS 2: Tablas de verdad. (5 ejercicios).

① 2 Variables.

$(P \wedge Q) \rightarrow P$ $2^2 = 4$ Filas.

P	Q	$P \wedge Q$	$(P \wedge Q) \rightarrow P$
V	V	V	V
V	F	F	V
F	V	F	V
F	F	F	V

→ Tautología

② $P \vee Q \leftrightarrow \neg P \wedge \neg Q$ $2^2 = 4$ Filas → Contradicción

P	Q	$\neg P$	$P \vee Q$	$\neg P \wedge \neg Q$	$P \vee Q \leftrightarrow \neg P \wedge \neg Q$
V	V	F	V	F	F
V	F	F	V	F	F
F	V	V	V	F	F
F	F	V	F	V	F

3 Variables.

③ $(P \vee \neg Q) \wedge R$ $2^3 = 8$ Filas. Contingencia.

P	Q	R	$\neg Q$	$P \vee \neg Q$	$(P \vee \neg Q) \wedge R$
V	V	V	F	V	V
V	V	F	F	V	F
V	F	V	V	V	V
V	F	F	V	V	F
F	V	V	F	F	F
F	V	F	F	F	F
F	F	V	V	V	V
F	F	F	V	V	F

④ $(p \vee q) \wedge \neg r$ $2^3 = 8$ Filas.

Contingencia

p	q	r	$p \vee q$	$\neg r$	$(p \vee q) \wedge \neg r$
V	V	V	V	F	F
V	V	F	V	V	V
V	F	V	V	F	F
V	F	F	V	V	V
F	V	V	V	F	F
F	V	F	V	V	V
F	F	V	F	F	F
F	F	F	F	V	F

⑤ 4 Variables.

$[(p \vee q) \vee r] \wedge s$ $2^4 = 16$ Filas.

Contingencia

p	q	r	s	$p \vee q$	$[(p \vee q) \vee r]$	$[(p \vee q) \vee r] \wedge s$
V	V	V	V	V	V	V
V	V	V	F	V	V	F
V	V	F	V	V	V	V
V	V	F	F	V	V	F
V	F	V	V	V	V	V
V	F	V	F	V	V	F
V	F	F	V	V	V	V
V	F	F	F	V	V	F
F	V	V	V	V	V	V
F	V	V	F	V	V	F
F	V	F	V	V	V	V
F	V	F	F	V	V	F
F	F	V	V	V	V	V
F	F	V	F	V	V	F
F	F	F	V	F	F	F
F	F	F	F	F	F	F

ITEMS 3: Simplificación de expresiones lógicas aplicando las leyes fundamentales. (5 ejercicios).

① Demostrar:

$$P \wedge P \equiv P$$

$$P \wedge P = (P \wedge P) \vee 0 \rightarrow \text{ley de identidad.}$$

$$= (P \wedge P) \vee (P \wedge \neg P)$$

$$= P \vee (P \wedge \neg P) \rightarrow \text{ley idem potencia.}$$

$$= (P \vee P) \wedge (P \vee \neg P) \rightarrow \text{ley distributiva.}$$

$$= P \wedge 1 \rightarrow \text{ley de identidad}$$

$$= \underline{P} \quad R//$$

$$\textcircled{2} [(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$$

$$\downarrow$$
$$[\neg(\neg(p \vee q \vee r)) \vee (\neg p \wedge \neg q \wedge \neg r)] \rightarrow \text{ley de morgan}$$

$$= 1 \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow \text{ley de complemento.}$$

$$= [(q \wedge (r \vee \neg r)) \vee (\neg q \wedge r)] \rightarrow \text{ley distributiva.}$$

$$= [q \wedge 1 \vee (\neg q \wedge r)] \rightarrow \text{ley de complemento.}$$

$$= [q \vee (\neg q \wedge r)] \rightarrow \text{ley de identidad}$$

$$= [(q \vee \neg q) \wedge (q \vee r)] \rightarrow \text{ley distributiva.}$$

$$1 \wedge (q \vee r) \rightarrow \text{ley de complemento.}$$

$$\underline{(q \vee r)} \quad R//$$

$$\textcircled{3} (P \wedge Q) \vee (P \wedge \neg Q)$$

$P \wedge (Q \vee \neg Q) \rightarrow$ ley de distribución

$P \wedge 1 \rightarrow$ ley de complemento.

$P \cdot 1 \rightarrow$ ley de identidad.

$P \text{ RII}$

$$\textcircled{4} (P \vee Q \vee R) \wedge (P \vee \neg Q \vee R)$$

$(P \vee R \vee Q) \wedge (P \vee R \vee \neg Q) \rightarrow$ ley de orden.

$(P \vee R) \vee Q \wedge (P \vee R) \vee \neg Q \rightarrow$ ley de distribución.

$(P \vee R) \vee (Q \wedge \neg Q) \rightarrow$ ley distributiva.

$(P \vee R) \vee 0 \rightarrow$ ley de complemento.

$P \vee R \text{ RII}$

$$\textcircled{5} (P \vee \neg P) \wedge Q$$

$1 \wedge Q \rightarrow$ ley de complemento.

Q

Items 4: Expresiones booleanas aplican en la evaluación de condiciones en programación (if-else, while; operadores lógicos en Python /C/ Java).

La expresión booleana permite la toma de decisiones en la programación evaluando las condiciones. Si es verdadero (True) o falso (False) para tomar control de la ejecución mediante estructuras condicionales.

If-else: Evalúa si alguna expresión booleana es verdadera, para luego ejecutar un bloque de código específico de lo contrario siendo falso omite el bloque.

Python

edad = 20

if edad >= 18:

print("Eres mayor de edad")

else:

print("Eres menor de edad")

While: Las expresiones booleanas. Son utilizadas para determinar si un bloque debe repetir continuamente. Siempre y cuando la expresión sea evaluada como true.

Operadores lógicos: Sirve para conectar múltiples expresiones booleanas para formar condiciones compuestas, se clasifican en tres:

AND: Verdadero si todas las expresiones son verdaderas.

OR: Verdadero si al menos una expresión es verdadera.

NOT: Se invierte el valor es decir verdadero pasa a falso o viceversa.

Ejemplo:

numero = int(input("Introduce un número entre 1 y 100:"))

El bucle sigue mientras el número está fuera del rango.

while número < 1 or número > 100:

print("Error: El número está fuera de rango.")

numero = int(input("Intentalo de nuevo:"))

print("¡Éxito! El valor es válido.")

Python: Utiliza las palabras clave True y False (mayúscula) lo cual considera 0 y estructuras vacías como False.

C/JAVA: Utilizan true y false (minúscula). En C, 0 es false y un número diferente de 0 es true. En Java, los operadores son AND, OR, NOT.

Universidad Nacional de Loja
APE 1 U1 Fase 2

Nombre: Willy Ramírez

Ciclo: 1A

Unidad: 1

Materia: Matemáticas Discretas

Fase 2: Construcción de Tablas de Verdad y Operaciones combinadas.
ITEMS 2: Tablas de verdad (5 ejercicios)

① 2 Variables

$(p \wedge q) \rightarrow p$ $2^2 = 4$ Filas

p	q	$p \wedge q$	$(p \wedge q) \rightarrow p$
v	v	v	v
v	f	f	v
f	v	f	v
f	f	f	v

→ Tautología

② $p \vee q \leftrightarrow \neg p \wedge \neg q$ $2^2 = 4$ Filas

p	q	$p \vee q$	$\neg p \wedge \neg q$
v	v	v	f
v	f	v	f
f	v	v	f
f	f	f	v

→ Contradicción

3 Variables

3. $(p \vee \neg q) \wedge r$ $2^3 = 8$ Filas

p	q	r	$(p \vee \neg q) \wedge r$
v	v	v	f
v	v	f	f
v	f	v	v
v	f	f	f
f	v	v	f
f	v	f	f
f	f	v	v
f	f	f	f

→ Contingencia

④ $(p \vee q) \wedge \neg r$ $2^3 = 8$ filas

$(p \vee q) \wedge \neg r$					
V	V	V	F	F	V
V	V	V	V	V	F
V	V	F	F	F	V
V	V	F	V	V	F
F	V	V	F	F	V
F	V	V	V	V	F
F	F	F	F	F	V
F	F	F	V	V	F

→ Contingencia

4 Variables

⑤ $[(p \vee q) \vee r] \wedge s$ $2^4 = 16$ Filas → Contingencia

$[(p \vee q) \vee r] \wedge s$					
V	V	V	V	V	V
V	V	V	V	V	F
V	V	V	V	F	V
V	V	V	V	F	F
V	V	F	V	V	V
V	V	F	V	V	F
V	V	F	V	F	V
V	V	F	V	F	F
F	V	V	V	V	V
F	V	V	V	V	F
F	V	V	V	F	V
F	V	V	V	F	F
F	V	F	V	V	V
F	V	F	V	V	F
F	F	F	F	F	V
F	F	F	F	F	F

ITEMS 3. Simplificación de expresiones lógicas aplicando las leyes fundamentales (5 ejercicios)

① Demostrar:

$$P \wedge P = P$$

$$P \wedge P = (P \wedge P) \vee 0 \rightarrow \text{Ley de identidad}$$

$$= (P \wedge P) \vee (P \wedge \neg P)$$

$$= P \vee (P \wedge \neg P) \rightarrow \text{Ley de idempotencia}$$

$$= (P \vee P) \wedge (P \vee \neg P) \rightarrow \text{Ley distributiva}$$

$$= P \wedge 1 \rightarrow \text{Ley de identidad}$$

$$= P. \quad \text{R//}$$

$$\textcircled{2} [(P \vee q \vee r) \vee (\neg P \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$$

$$[\neg(\neg(P \vee q \vee r)) \vee (\neg P \wedge \neg q \wedge \neg r)] \rightarrow \text{Ley de Morgan}$$

$$= 1 \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow \text{Ley de complemento}$$

$$= [(q \wedge (r \vee \neg r)) \vee (\neg q \wedge r)] \rightarrow \text{Ley distributiva}$$

$$= [q \wedge 1 \vee (\neg q \wedge r)] \rightarrow \text{Ley de complemento}$$

$$= [q \vee (\neg q \wedge r)] \rightarrow \text{Ley de identidad}$$

$$= [(q \vee \neg q) \wedge (q \vee r)] \rightarrow \text{Ley distributiva}$$

$$= 1 \wedge (q \vee r)$$

$$= (q \vee r) \quad \text{R//}$$

③ $(P \wedge Q) \vee (P \wedge \neg Q)$

$P \wedge (Q \vee \neg Q) \rightarrow$ Ley de distribución

$P \wedge 1 \rightarrow$ Ley de complemento

$P //$

④ $(P \vee Q \vee R) \wedge (P \vee \neg Q \vee R)$

$(P \vee R \vee Q) \wedge (P \vee R \vee \neg Q) \rightarrow$ Ley orden

$(P \vee R) \vee Q \wedge (P \vee R) \vee \neg Q \rightarrow$ Ley distribución

$(P \vee R) \vee (Q \wedge \neg Q) \rightarrow$ Ley distributiva

$(P \vee R) \vee 0 \rightarrow$ Ley de complemento

$P \vee R //$

⑤ $(P \vee \neg P) \wedge Q$

$1 \wedge Q \rightarrow$ Ley de complemento

$Q //$

La expresión booleana permite la toma de decisiones en la programación, evaluando las condiciones si es verdadero (True) o falso (False) para tomar control de la ejecución mediante estructuras condicionales

```
edad = 20
```

```
if edad >= 18;  
    print("Eres mayor de edad");
```

```
else:  
    print("Eres menor de edad");
```

If-else Evalúa si alguna expresión booleana es verdadera para luego ejecutar un bloque de código específico de lo contrario siendo falso omite el bloque

While: Las expresiones booleanas son usadas para determinar si un bloque debe repetirse continuamente siempre y cuando la expresión sea evaluada como true.

Operadores lógicos Sirve para conectar múltiples expresiones booleanas para formar condiciones compuestas, se clasifican en tres:

AND: Verdadero si todas son verdaderas

OR: Verdadero

NOT: Se invierte el valor, es decir 1 es 0 y 0 es 1

```
numero = int(input("Introduce un número entre 1 y 100:"))  
# El bucle sigue mientras el número esté fuera del rango  
while numero < 1 or numero > 100:  
    print("Error: El número está fuera de rango.")  
    numero = int(input("Intentalo de nuevo: "))
```

```
print("¡Éxito! El número es válido.")
```

Python: Usa las palabras clave True y False (mayúsculas) lo cual considera 0 y estructuras vacías como false.

C/Java: Usan true y false. En C, 0 es false y un número diferente de 0 es true. En Java, los operadores son && (AND), || (OR), ! (NOT)

UNL

- Estudiante: Alison Cristina Ramirez
- Asignatura: Matemáticas Discretas
- Título: Construcción de tablas de verdad

- Docente:
- Ciclo:
- Unidad:

FASE 2;

Item 2; Tabla de verdad (5 ejercicios)

- ① 2 variables

$$(p \wedge q) \rightarrow p \quad 2^2 = 4 \text{ filas}$$

p	q	$p \wedge q$	$(p \wedge q) \rightarrow p$
V	V	V	V
V	F	F	V
F	V	F	V
F	F	F	V

→ TAUTOLOGÍA

- ② $p \vee q \leftrightarrow \neg p \wedge \neg q \quad 2^2 = 4 \text{ filas}$

p	q	$\neg p$	$\neg q$	$\neg p \wedge \neg q$
V	V	F	F	F
V	F	F	V	F
F	V	V	F	F
F	F	V	V	V

→ Contradicción

- ③ 3 variables. → $(p \vee \neg q) \wedge r \quad 2^3 = 8 \text{ filas}$

q	p	$\neg q$	$(p \vee \neg q) \wedge r$
V	V	F	V
V	V	F	F
F	V	V	V
F	V	V	F
V	F	F	F
V	F	F	F
F	F	V	V
F	F	V	F

→ CONTINGENCIA

④ $(P \vee Q) \wedge \neg R$

$2^3 = 8$ filas

R	$(P \vee Q)$	$\neg R$
V	V	F
F	V	V
V	V	F
F	V	V
V	F	F
F	F	V
V	F	F
F	F	V

→ CONTINGENCIA

⑤ 4 variables

$[(P \vee Q) \vee R] \wedge S$

$2^4 = 16$ filas

$(P \vee Q)$	$\vee R$	$\wedge S$
V	V	V
V	V	F
V	V	V
V	V	F
V	F	V
V	F	F
V	F	V
V	F	F
F	V	V
F	V	F
F	V	V
F	V	F
F	F	V
F	F	F
F	F	V
F	F	F

→ CONTINGENCIA

Item 3. Simplificación de expresiones lógicas aplicando las leyes fundamentales (5 ejercicios)

• ① Demostrar

$$P \wedge P = P$$

$$P \wedge P = (P \wedge P) \vee 0 \rightarrow \text{Ley de identidad}$$

$$= (P \wedge P) \vee (P \wedge \neg P)$$

$$= P \vee (P \wedge \neg P) \rightarrow \text{Ley de Idempotencia}$$

$$= (P \vee P) \wedge (P \vee \neg P) \rightarrow \text{Ley distributiva}$$

$$= P \wedge 1 \rightarrow \text{Ley de identidad}$$

$$= P // Rta$$

• ② $[(P \vee q \vee r) \vee (\neg P \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$

$$[\neg(\neg(P \vee q \vee r)) \vee (\neg P \wedge \neg q \wedge \neg r)] \rightarrow \text{Ley de Morgan}$$

$$= 1 \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow \text{Ley distributiva}$$

$$= [(q \wedge 1) \vee (\neg q \wedge r)] \rightarrow \text{Ley de complemento}$$

$$= [q \vee (\neg q \wedge r)] \rightarrow \text{Ley de identidad}$$

$$= [(q \vee \neg q) \wedge (q \vee r)] \rightarrow \text{Ley distributiva}$$

$$= 1 \wedge (q \vee r)$$

$$= (q \vee r) // Rta$$

• ③ $(P \wedge q) \vee (P \wedge \neg q)$

$$= P \wedge (q \vee \neg q) \rightarrow \text{Ley distributiva}$$

$$= P \wedge 1 \rightarrow \text{Ley de complemento}$$

$$= P \cdot 1 \rightarrow \text{Ley de identidad}$$

$$P // Rta$$

• ④ $(P \vee q \vee r) \wedge (P \vee \neg q \vee r)$

$$= (P \vee r \vee q) \wedge (P \vee r \vee \neg q) \rightarrow \text{Orden}$$

$$= (P \vee r) \vee q \wedge (P \vee r) \vee \neg q \rightarrow \text{Ley distributiva}$$

$$= (P \vee r) \vee (q \wedge \neg q) \rightarrow \text{Ley distributiva}$$

$$= (P \vee r) \vee 0 \rightarrow \text{Ley de complemento}$$

$$= P \vee r // Rta$$

• ⑤ $(P \vee \neg P) \wedge q$

$$= 1 \wedge q \rightarrow \text{Ley de complemento}$$

$$= q // Rta$$

Item 4. Expresiones booleanas aplican, en la evaluación de condiciones en programación.

La expresión booleana permite la toma de decisiones en la programación evaluando las condiciones si es verdadero (True) o falso (False) para tomar control de la ejecución mediante estructuras condicionales.

edad = 20

```
if edad >= 18:
    print("Eres mayor de edad");
```

```
else:
    print("Eres menor de edad");
```

if-else: Evalúa si alguna expresión booleana es verdadera para luego ejecutar un bloque de código específico de lo contrario siendo falso omite el bloque.

While: Las expresiones booleanas son usadas para determinar si un bloque debe repetirse continuamente siempre y cuando la expresión sea evaluado como True.

Operadores lógicos: Sirve para conectar múltiples expresiones booleanas para formar condiciones compuestas se clasifican en tres:

And: Verdadero si todas son verdaderas

Or: Verdadero

Not: Se invierte el valor; si es 1 se convierte en 0 y viceversa.

```
boolean terminado = false;
```

```
int intentos = 0;
```

```
while (!terminado) {
```

```
    intentos ++
```

```
    if (intentos >= 3) {
```

```
        terminado = true // Cambia el estado para salir.
```

```
        System.out.println("Intento: " + intentos);
```

```
    }
```

Python: Usa las palabras clave True y False (mayúsculas) lo cual considera 0 y estructuras vacías como False.

C/Java: Usan true y false. En C, 0 es false y un número diferente de 0 es true. En Java los operadores son & & (AND), || (OR), ! (NOT).



Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación
